

Reliability of Microscaling-Format Neural Networks under Bit Faults

Peeyush Agarwal

2 October 2026

1 Background and theory

Low-precision inference is limited by dynamic range: a 4- or 6-bit number cannot span the spread of values in a weight tensor. Microscaling (MX) formats solve this by sharing one scale across a small block. A block of K elements is stored as one 8-bit exponent X (the E8M0 scale) together with K narrow element codes m_i , and the hardware reconstructs

$$v_i = m_i \cdot 2^{X-127}, \quad i = 1, \dots, K. \quad (1)$$

An element code is a small sign-exponent-mantissa number decoding to $(-1)^s(1+f/2^M)2^{e-\text{bias}}$ for $e > 0$; e4m3 uses 1+4+3 bits, e3m2 1+3+2, e2m1 1+2+1. The two 8-bit formats reserve codes for NaN and infinity, whereas the 6- and 4-bit formats have none, so every one of their bit patterns is a finite number. That distinction turns out to matter more than any other. The scale follows the OCP rule $X = \lfloor \log_2 \max_i |v_i| \rfloor - e_{\max}$, with $e_{\max}$ the exponent of the largest value the element format can hold.

In ordinary floating point a value is a self-contained word, so a bit flip perturbs one number. Equation (1) breaks that. An *element* fault flips a bit of one m_i : a mantissa bit changes the value by at most a relative 2^{j-M} , an exponent bit multiplies it by $2^{\pm 2^j}$, and the sign bit negates it. A *scale* fault flips a bit of X , so $X \rightarrow X \pm 2^j$ and all K elements are multiplied by $2^{\pm 2^j}$ at once; if the flip produces the reserved code 255 the whole block becomes NaN. The number of values a fault alters is its *blast radius*: 1 for an element fault, K for a scale fault.

This predicts something before any measurement. A block occupies $8 + Kw$ bits (w = element width), of which only 8 belong to the scale, about 3% for $K = 32, w = 8$. Yet if every stored bit is equally likely to flip, the expected number of corrupted *values* per block is $8K$ from scale faults against Kw from element faults, a ratio of $8 : w$. An 8-bit format therefore takes about half its corrupted values through a scale occupying 3% of storage, and a 4-bit format two thirds.

2 Fault model and metrics

I use the single-bit-flip model standard in fault-injection work, applied to stored values: one bit of one element code, or of one scale byte, is inverted (Section 6 shows that a second flip in the same word adds nothing first-order). Weight faults are persistent and affect every inference; activation faults are transient and affect only the inference in which they occur. Let $D = \{(x_i, y_i)\}_{i=1}^N$ be a fixed evaluation set, $\hat{y}_i$ the prediction of the fault-free but MX-quantised network, and $\hat{y}_i^\varphi$ the prediction under fault φ .

metric	definition	meaning
corruption count	$c(\varphi) = \sum_i \mathbb{K}[\hat{y}_i^\varphi \neq \hat{y}_i]$	inferences the fault changed
SDC indicator	$s(\varphi) = \mathbb{K}[c(\varphi) \geq 1]$	did it corrupt anything at all
SDC rate	$\hat{p} = \Phi ^{-1} \sum_{\varphi \in \Phi} s(\varphi)$	fraction of faults that corrupt
per-inference rate	$r(\varphi) = c(\varphi)/N$	chance a given inference is corrupted
non-finite rate	$ \Phi ^{-1} \sum_{\varphi} \mathbb{K}[\text{NaN/Inf output}]$	faults that destroy the output

The SDC rate counts a fault as a failure if it changes even one prediction, so it is deliberately sensitive and comparable across configurations rather than across papers. Weight and activation faults must not be compared on it, since a weight fault is exposed to all N inferences and an activation fault to one; the comparable quantity is $r(\varphi)$. Rates carry Wilson 95% intervals. Because the element alphabets hold at most 256 codes, the decoded value of every (code, bit) pair is tabulated exactly rather than sampled.

3 Method

The campaign compares a golden run against a faulty one. The network is first put in the form an accelerator executes: batch normalisation folded into the preceding convolution, and RepVGG’s branches fused. This matters because an unfolded batch norm re-normalises whatever the corrupted convolution produced and hides

Algorithm 1 Fault-injection campaign

Require: network M , format F , block size K , evaluation set D , budget n

```
1:  $M \leftarrow \text{DEPLOY}(M)$  ▷ fold batch norm, fuse branches
2: for all quantisable layers  $\ell$  do
3:   view  $W_\ell$  as (outputs  $\times$  reduction), split the reduction axis into blocks of  $K$ 
4:    $X_\ell \leftarrow \lfloor \log_2 \max |W_\ell| \rfloor - e_{\max}$  per block;  $C_\ell \leftarrow \text{round}_F(W_\ell 2^{-X_\ell})$ 
5:    $W_\ell \leftarrow \text{DECODE}(X_\ell, C_\ell)$  ▷ the network now holds MX values
6: end for
7:  $\hat{y} \leftarrow \arg \max M(D)$  ▷ golden predictions
8: for  $t = 1$  to  $n$  do
9:   draw  $\varphi = (\ell, \text{site}, \text{index}, b)$  uniformly over all stored bits
10:  if site = element then  $C_\ell[\text{index}] \leftarrow C_\ell[\text{index}] \oplus 2^b$ ; update that one weight
11:  else  $X_\ell[\text{index}] \leftarrow X_\ell[\text{index}] \oplus 2^b$ ; update the  $K$  weights of the block
12:  end if
13:   $\hat{y}^\varphi \leftarrow \arg \max M(D)$ ; record  $c(\varphi)$ ,  $s(\varphi)$ , non-finiteness; restore weights
14: end for
15: return rates over  $\Phi$  with Wilson intervals
```

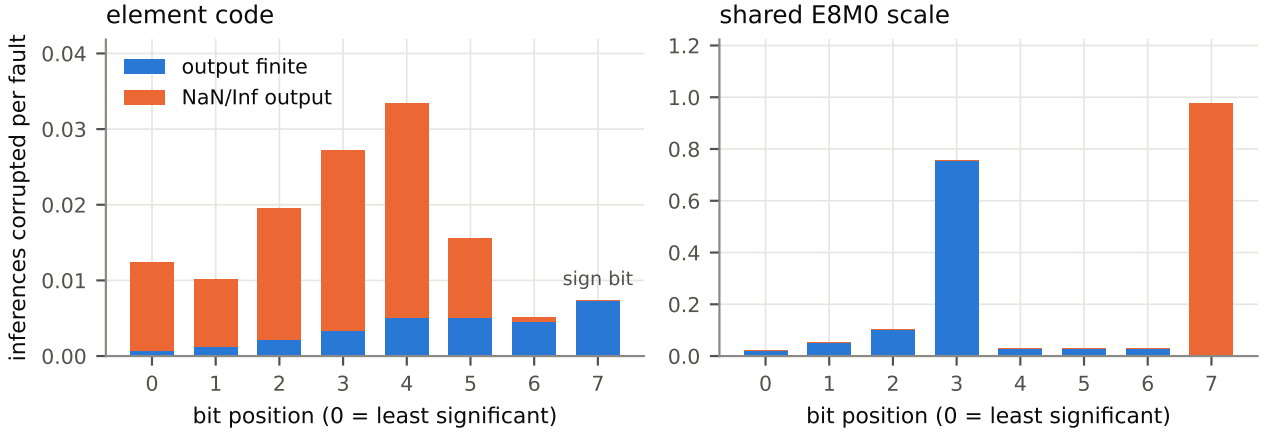


Figure 1: Exhaustive per-bit damage per inference (ResNet8, e4m3, $K = 32$, 638,624 injections), split by whether the output stayed finite. The sign bit does the most finite damage, but most element damage is flips into the NaN code, which a sign flip cannot make. Scale bits 3 and 7 are almost never set, so flipping them grows the block $256\times$ or overflows it; bits 4–6 are always set, and flipping them shrinks it.

part of the damage. Weights are then encoded into real MX bit patterns, the fault is applied *in the code domain*, and the block is decoded only afterwards, which is what lets a scale fault perturb all K values.

Line 8 defines the sampling. Drawing bits uniformly is the physically faithful choice, since faults land in proportion to storage; replacing that line by an enumeration of every addressable bit makes the campaign *exhaustive*, giving the true failure rate with no sampling error.

Setup. CIFAR-10 with ResNet8 (78k parameters, 87.0% top-1), RepVGG-A0 (7.0M, 91.5%, three seeds) and a vision transformer (2.7M, DeiT-Tiny width, 81.9%, three seeds), plus ResNet8 at five widths from 20k to 692k with ten independently trained models each; the three architectures were also retrained on CIFAR-100. Formats e4m3, e5m2, e3m2, e2m3, e2m1, block sizes 8–64. The evaluation set is a fixed class-balanced subset of 200 test images; campaigns use 3000 injections unless stated otherwise.

4 Results

Exhaustive ground truth. Flipping all 638,624 bits of ResNet8’s MXFP8 weight storage gives a true SDC rate of 0.4171. The ordinary 3000-injection campaign returned 0.4270 with interval $[0.4094, 0.4448]$: 2.4% off in relative terms, interval containing the truth. Repeating the enumeration under e3m2 gives a second exact reference (483,904 injections, rate 0.1079) and a second confirmation of the sampling (estimate 0.1070, interval $[0.0964, 0.1186]$). The two enumerations turned out to have been recorded from two different trained copies of ResNet8, which is why they are used below as two separate references and never as a comparison between formats. Exactly, under e4m3 element faults corrupt at 0.4003 and scale faults at 0.9442, with 3.27 against 49.89 of 200 inferences corrupted. Vulnerability runs opposite to layer size: the 3584-bit stem is the most fragile layer at 0.853, the 304,128-bit final convolution the most robust at 0.289, so a uniform campaign spends 48%

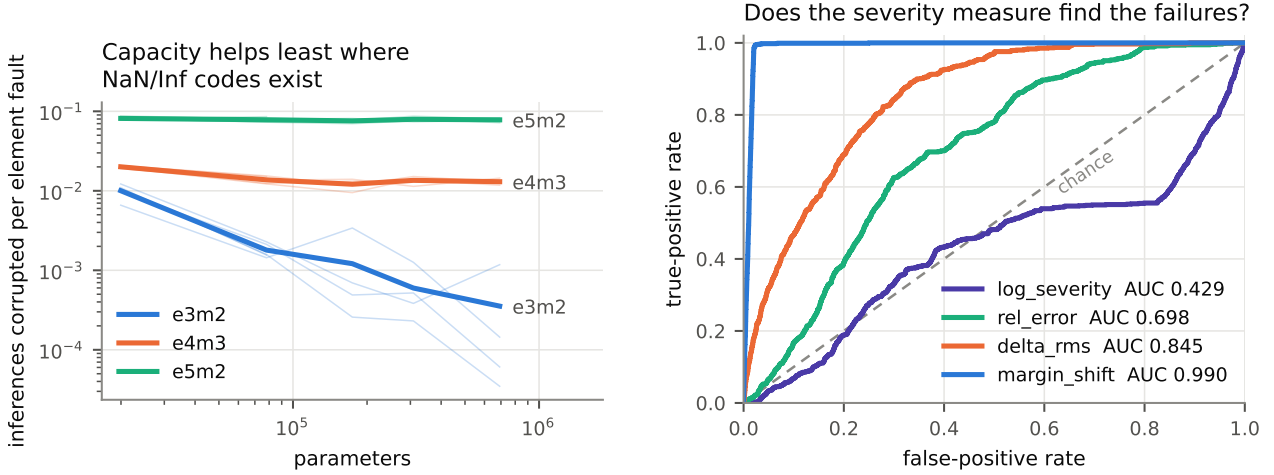


Figure 2: Left: the fraction of inferences an element fault corrupts, against parameter count, with every independently trained model drawn faintly behind each mean. Capacity helps least for the format with the most NaN/Inf codes, and not at all for e5m2. Right: how well each severity measure ranks the faults that actually failed. The measure proposed in the project plan falls below the chance diagonal; margin_shift reaches AUC 0.990.

of its budget on the most robust layer. Per bit (Figure 1), 1.27% of element faults flip a code into NaN; each corrupts every inference, and together they carry 78% of all element damage.

Accuracy does not predict reliability, for two separate reasons. On the transformer every format except MXFP4 lands within 0.2 points of fp32 accuracy, so e5m2 and e3m2 differ by 0.01 points – and the fraction of inferences their element faults corrupt is 0.0690 [0.0598, 0.0782] against 0.00003 [0.00001, 0.00004]. Rescoring all 119 campaigns this way, 1,441,528 recorded faults over eight models, the ordering $e3m2 < e4m3 < e5m2$ holds on 8 of 8 models with disjoint intervals throughout, against 3 of 8 under the any-image rate. Two independent mechanisms produce the gaps.

Special codes. Only formats owning NaN/Inf codes can produce a non-finite value, and nothing downstream absorbs one. On the transformer 94% of e5m2’s element failures were non-finite events, and on RepVGG 92%, against none for e3m2 in any campaign or in the 483,904-fault exhaustive enumeration; per inference the share rises to 96.9%, since the metric counts how much damage each fault does and a NaN corrupts every inference it touches. Training ResNet8 at five widths, ten seeds each, isolates this: the non-finite rate is a property of the format, varying under 20% across a 35-fold change in capacity with no correlation against model size, while the benefit a format draws from capacity falls with its number of special codes (Figure 2, left). Per inference that dose-response is nearly exact – over a 35-fold capacity range the format with no special codes improves $25\times$, the one with two improves $1.5\times$, and the one with eight does not improve measurably ($0.96\times$). The pathway is cheap to close: decoding special element codes as zero on read turns each such fault into an ordinary single-weight perturbation. On RepVGG and the transformer, where those faults carry 97–100% of element damage, this cuts the element per-inference rate by at least $29\text{--}4100\times$ (95% bound, three seeds each), to roughly the level of e3m2; on ResNet8, by $4\text{--}53\times$.

Sensitivity, and what it is not. A second mechanism appears among formats with no special codes at all. Perturbing single weights by 1.5 layer-RMS, a probe with no bit patterns, never flipped a prediction of the e3m2 and e5m2 networks in 1500 trials but flipped e4m3’s 2–5% of the time. The fragility belongs to the network, not the encoding, and its cause is the sensitivity $s_i = |\partial m / \partial w_i|_{\text{rms}} / m$, with m the margin between the top two logits: a perturbation of $\delta \cdot \text{rms}$ flips a prediction when $s_i \delta > 1$, and computing s by backpropagation reproduces the probe with correlation +0.998. Across the five formats s predicts the failure rate at $r = +0.925$ (+0.928 per inference on perturbation-driven faults, unchanged by the metric). It is *not* mantissa width, which correlates at -0.251 once MXFP4 is included – e2m1 has the fewest mantissa bits and the highest failure rate.

Pressed further, s turns out to measure something narrower than it promises. Decomposing it into its two factors over 200 images, the gradient magnitude $|\partial m / \partial w|_{\text{rms}}$ agrees across all five formats to within $1.08\times$ at both the median and the 99th percentile; the whole 10.3-fold spread in s , and 22.2-fold at the 99th percentile, comes from the margin in its denominator, because s is defined as a maximum over images and the closest-to-the-boundary image varies 33-fold. Alignment is refuted by a permutation that preserves both marginals and moves the tail at most 2%. And the margin that does the work belongs to the evaluation set rather than to the network. Measured on the campaigns’ own 200 images, $1/\min m$ ranks fifteen networks by failure rate at Spearman +0.98; measured on 2000 fresh images it ranks them at +0.21.

Shared scales dominate on every architecture. Scale faults outweigh element faults by $2.4\times$ on ResNet8 (exhaustive, e4m3), $28.7\times$ on RepVGG-A0 and $12.1\times$ on the transformer, and they corrupt 16–50 of 200 inferences each. Per inference the dominance is larger still and never reverses on any model or format: $13\times$ for e4m3 on ResNet8-w16, of order 10^3 for e3m2 on RepVGG and the transformer, where the denominator is statistically indistinguishable from zero and the ratio should be read as a lower bound. Only for e5m2 does it shrink, to $2.6\times$, because that format’s element faults already corrupt everything through the NaN pathway. This is the most robust design conclusion in the study. Scale faults also produce non-finite outputs at 7–13% *for every format*, including those with no special element codes, because the E8M0 scale has its own NaN code and an MSB worth 128 exponent steps.

Per fault, activations are about as vulnerable as weights. On the comparable per-inference rate an element fault does $0.6\text{--}2.2\times$ as much damage in a weight as in an activation across e4m3 and e5m2 on the three CIFAR-10 models, and a scale fault $0.9\text{--}1.6\times$, for the same reason: in either, the faults that land on a special code do most of the damage. An earlier version found weights $5\text{--}9\times$ worse; its codec mapped NaN inputs to zero, so every re-quantised layer erased the NaN an activation fault had produced. What separates the two is exposure: a weight fault persists, an activation fault lasts one inference.

A harder task. Retrained on CIFAR-100 (ViT 47–49%, RepVGG 70–71%), every headline replicates: equal-accuracy formats, the ordering $e3m2 < e4m3 < e5m2$ on all nine networks, scale dominance, and NaN codes carrying 94–100% of e4m3/e5m2 element damage on the larger models. What changes is the floor: lower margins make the formats without special codes $3\text{--}22\times$ more vulnerable while e4m3 and e5m2 barely move, so the e4m3/e3m2 gap on the transformer shrinks from $461\times$ to $19\times$.

Block size, and how much one model can be trusted. Raising K from 8 to 64 cuts the storage share of scale bits from 11.1% to 1.7% while a scale fault’s SDC rate rises from 0.731 to 0.941, with accuracy nearly flat in K : a clean efficiency-against-reliability trade. Across ten seeds per width, the spread between independently trained models is about ten times the uncertainty contributed by 3000 injections, so a reliability figure quoted from one trained model is far less certain than its injection count suggests.

5 Three problems with the proposed methodology

The severity measure, and a replacement. The plan scores an element fault by $d = |\log_2 |v'| - \log_2 |v||$. Over 39,337 element faults its rank correlation with observed SDC is -0.035 and its AUC is 0.435 – it ranks failures slightly *worse* than chance (Figure 2, right). The reason is in the definition: negating a value leaves $|v|$ unchanged, so a sign flip scores zero severity although the sign bit does more perturbation damage than any other element bit. Three successively better measures follow: relative error (0.699), absolute change in layer-RMS units (0.845), and $\text{margin_shift} = s_i |v' - v| / \text{rms}$, the fraction of the decision margin a fault consumes, at AUC 0.990. Its threshold needs no tuning: flagging faults with $\text{margin_shift} > 1$ catches 98.5% of all failures while flagging 4.4% of the fault space, which is directly usable for the value-aware sampling the plan wanted. It also carries a caveat the original formulation hides: severity is not a property of the number system, since the same flip in the same format behaves very differently in two networks of equal accuracy.

What the replacement buys. A better ranking is only worth having if it saves injections, so both exhaustive campaigns were reused as ground truth and sampling strategies replayed against them offline: score the whole fault space, stratify at quantiles of margin_shift (six strata from the 50th to the 99.5th percentile, free since the score covers the whole space), spend a fifth of the budget on a pilot and Neyman-allocate the rest. Because the per-inference rate is a mean of per-fault proportions rather than a proportion, each stratum’s variance is estimated by the pilot rather than assumed as $p(1-p)$. The target is heavily zero-inflated – 89.2% of ResNet8’s e3m2 faults corrupt no inference at all – and the score’s extreme percentiles isolate the thin tail that holds nearly all the variance: the estimator’s variance falls $57\text{--}120\times$ on both formats against uniform sampling at equal budget, unbiased at every budget. Fixed cuts chosen earlier for the any-image rate give only $1.5\text{--}8.3\times$ on this target.

The standard scale rule distorts the block-size trend. Because the largest representable element lies below the top of the binade the OCP rule aligns to, the block maximum is clipped, and smaller blocks mean more maxima and more clipping. Injecting an identical set of faults at every block size, element-side any-image SDC varies by a factor 2.21 on ResNet8 and 7.25 on RepVGG-A0 under the OCP rule against 1.04 and 1.48 under a rule that cannot clip, which made the trend look manufactured. Per inference the two rules differ by only $1.24\times$, and on ResNet8 the non-clipping rule gives the *larger* spread. What the per-inference data show

under both rules is a genuine monotone effect, larger blocks corrupting fewer inferences, whose mechanism this study does not establish. So a non-clipping control remains necessary – the OCP rule really does distort the trend it is measured under – but the trend itself is not an artefact.

The usual SDC definition hides the effect it is used to measure. A fault is conventionally counted as causing SDC if *any* of the n evaluated inferences changes its prediction. That quantity grows with n and saturates, so beyond a point it measures whether the evaluation set contains an input near a decision boundary rather than anything about the fault. Replaying one fixed list of 1500 element faults against nine class-balanced sets of 200 images, on each of two independently trained ResNet8s, the any-image rate moves by up to 7.0 times the width of the Wilson interval that 1500 injections earns, ranked by each set’s smallest margin at Spearman +0.80 to +0.93. Under it, e3m2 and e4m3 are indistinguishable ($1.03\times$ and $0.93\times$) and their order reverses from one image set to the next. Scored instead by the per-inference rate – what fraction of the n inferences each fault corrupts – the same faults on the same folds give $5.57\times$ and $7.64\times$ between the two formats, with the e4m3 figure varying only $1.15\text{--}1.27\times$ across image sets. The mantissa effect is real; the any-image metric was destroying it, and the per-inference rate belongs in the primary column.

A related trap: training is not bit-reproducible across machines, and the two exhaustive enumerations turned out to come from two *different* networks of equal accuracy. Replaying a few hundred recorded faults against the checkpoint catches this.

6 The plan’s two open design choices

Single-bit injection is sufficient. The plan adopted single-bit flips and left its multi-bit H-model as an extension. To settle it, 2000 stored words per format were sampled on each of two ResNet8s and *every* single- and double-bit pattern of each word was injected (390,000 injections), recording which images each fault corrupted. A double flip corrupts $1.04\text{--}1.55\times$ as many inferences as a single flip, and an adjacent pair – the pattern a multi-cell upset produces – $0.85\text{--}1.13\times$; the format ordering and scale dominance are unchanged. Doubles are sub-additive, doing 52–85% of the damage of the union of their two single flips: a second flip moves a word off the NaN code its partner would have reached, and two scale bits of opposite value partly cancel. So multi-bit outcomes cannot be synthesised from single-bit data – but they are not needed. Under the H-model’s independent flips, doubles carry 1% of a word’s expected damage only once the per-bit flip probability reaches $2\text{--}3 \times 10^{-3}$; at that rate ResNet8 already holds over a thousand faulty bits at once, so the one-fault-per-inference premise of every campaign fails through accumulation across words three orders of magnitude in p before multiplicity within a word matters.

Enumerate impact exactly; do not learn it. The plan also asked whether impact should be characterised as TreeFI does, by value intervals that a regression tree learns from a pilot of injections, or by exact enumeration over the MX code table, which gives every fault’s decoded change $|v' - v|$ in closed form. Replayed against both exhaustive campaigns at equal budgets, strata from exact enumeration of $|v' - v|$ alone – no gradient, no injections – cut variance $48\text{--}65\times$ against uniform sampling. Strata learned from a 20% pilot never beat uniform sampling on either format, at 6 or 16 leaves, with or without the layer as a feature ($0.10\text{--}0.98\times$). Two things defeat the learner. A pilot of a few hundred faults, on a target where 89% of faults do nothing, sees a handful of failures – too few to place the strata, although a tree fitted to the *whole* truth reaches $53\text{--}102\times$ on e3m2, so the information is in the features. And on e4m3 the tail is not a value range at all but the discrete set of (code, bit) pairs that land on NaN, which a partition of value space spreads across every leaf and a code table lists exactly. Interval learning answers a problem – an alphabet of 2^{32} values – that MX does not have.

7 Status

The study covers three architecture families, answers all five objectives, and settles both of the plan’s open design choices. Design guidance follows directly: prefer element formats without NaN/Inf codes, or guard the special codes on read; protect the shared scale before anything else, in weights and activations alike; and expect a coarser mantissa to help – e3m2 corrupts $5.6\text{--}7.6\times$ fewer inferences than e4m3 on two independently trained networks – though not by field width: MXFP4 ranks worst on CIFAR-10, best on CIFAR-100. For campaigns: inject single-bit faults, score them per inference, and stratify on exact per-code enumeration of the decoded change, weighted by sensitivity where a gradient is available, for $57\text{--}120\times$ fewer injections at a given precision.

Re-measurement revised several earlier claims – the ranking does not reverse between architectures, weights are not more vulnerable than activations, the sign bit is worst only for perturbation damage – and `docs/findings.md` records each. ImageNet is unavailable here, so CIFAR-100 stands in: a harder task, not ImageNet.